Agentic AI System Design Report

The Run Steward: a Bounded Single Agent for a Pre-Registered Computational-Chemistry Campaign

Frederic Petrignani

Udacity AI Mastery Capstone — Module 07: Design of Autonomous and Semi-Autonomous Agentic Workflows

2026-10-17

Agentic_AI_System_Design_Report.pdf

Overview

This report describes an agentic workflow built for a real, running research campaign: the delta-probed infrared pipeline of the author's capstone plan, whose calculations run on five rented machines and a laptop under written, pre-registered rules. The agent — the run steward — observes machines and result files, proposes exactly one next action that cites a rule, passes the proposal through a deterministic gate, acts through 11 allow-listed tools, and records every step in an append-only ledger. It is a single agent with one reasoning loop, implemented as a LangGraph state graph with a language-model reasoning node (a LangChain chat model: the Anthropic API or an OpenAI-compatible endpoint such as the course's Vocareum keys, chosen per run and logged with every proposal) and a deterministic reference policy used for testing. An agentic approach fits because the decisions are sequential, depend on state that changes on its own and require tools; a bounded design fits because the correct behaviour is rule application under uncertainty, and the cost of a wrong autonomous action — a deleted server, a duplicate forty-hour run, a false scientific claim — far exceeds the cost of asking a human (Amodei et al., 2016; Shavit et al., 2023).

For a reader without chemistry: The project's calculations run for days on rented computers, and things go wrong at night: a job dies, a lock file blocks the next one, a log falls silent while the work continues. The run steward is a small program (an 'agent') that watches those machines, applies a written table of rules distilled from the project's own past mistakes, and may take only a short list of harmless actions; anything that costs money, deletes data or makes a scientific claim it hands to a human. The scenarios in section 4 replay real log excerpts to check that it behaves. Terms are defined in https://github.com/thebreadishard/udacity-capstone-plan/blob/master/plans/05_delta-probed-ir-pipeline/modules/GLOSSARY.md.

System Architecture and Design

Components. The persona is a careful laboratory technician who applies written rules and escalates what matches no rule. The reasoning loop is observe → propose → gate → act → record, with a conditional edge back to observe until a terminal action (wait or escalate) or a step budget. Memory has three parts: a curated long-term rule table of 41 rules (29 enforced mechanically by the gate, the rest shaping the reasoning prompt), a campaign state, and a rolling window of observations and actions taken in the run. Tools are the project's own status reader, launcher, result readers, ledger and notification channel, wrapped behind an interface whose replay implementation substitutes log fixtures for machines. The design follows the reasoning-and-acting pattern of ReAct (Yao et al., 2023) and the tool-use framing of Toolformer (Schick et al., 2023), with one deliberate departure: the model never executes; it proposes a structured object {action, args, rule_id, reason} and plain code decides. Figure 1 shows the graph; the notebook draws it from the compiled LangGraph object.

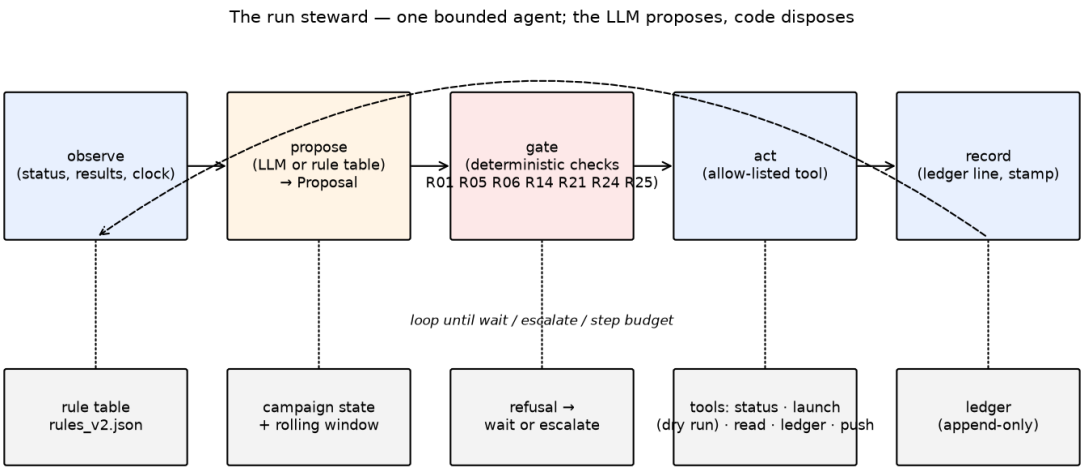


Figure 1. The run steward: one bounded agent. The reasoning node proposes; the deterministic gate disposes; every step is recorded.

Design choices and tradeoffs. Structured output with a cited rule identifier makes every decision auditable against the table. A closed allow-list means that a wrong or manipulated proposal cannot reach anything outside eleven actions, none of which creates or deletes machines, spends money, publishes, changes a rule or passes a verdict. The rule table limits what the agent handles: anything novel escalates. That is a capability tradeoff made on purpose for a campaign where a human question is cheap and a wrong action is not; the survey literature on LLM-based agents lists exactly this tension between autonomy and controllability (Wang et al., 2024; Xi et al., 2023).

Decision Logic and Behavior

Decision logic has two layers. The reasoning layer receives the persona, the allow-list, the whole rule table, the actions already taken and the last observations (verbatim log excerpts with the facts a tool extracted) and returns one proposal. The gate layer is deterministic and checks, in order: that the action is allow-listed and cites a rule that exists; that no observation contains an instruction addressed to the agent (if one does, only escalation is allowed); that a launch has a passed dry run of the same job and no live instance; that a lock is removed only when its recorded process is dead on the same host and the process list has been checked with a pattern that cannot match the checking shell itself; that a statistic of the form 'the model respects X' is recorded only beside the same statistic on the target; that a training job whose checkpoint is fresh is not restarted; that arguments never touch a human-gated action; and that the run's launch budget is not exhausted. A refused proposal is replaced by the safe substitute the rule names — a dry run, a process check, a wait or an escalation — never by nothing.

Table 1. The 17 pre-registered scenarios (S1–S8: real log excerpts of 25 September 2026; S9–S17: the records of 27 September – 3 October) replayed with the deterministic reference policy: 17 of 17 correct.

scenario	actions taken	rules cited	gate substitutions	escalated
S1	check_runner → remove_stale_lock → dry_run → launch → wait	R06 R05 R01 R01 R06	—	no
S2	launch → wait	R01 R06	—	no

S3	record_ledger → wait	R13 R22	—	no
S4	escalate_to_human	R14	—	yes
S5	wait	R21	—	no
S6	escalate_to_human	R25	—	yes
S7	escalate_to_human	R32	—	yes
S8	escalate_to_human	R24	—	yes
S9	escalate_to_human	R33	—	yes
S10	escalate_to_human	R34	—	yes
S11	escalate_to_human	R35	—	yes
S12	escalate_to_human	R36	—	yes
S13	escalate_to_human	R37	—	yes
S14	escalate_to_human	R38	—	yes
S15	escalate_to_human	R39	—	yes
S16	escalate_to_human	R40	—	yes
S17	escalate_to_human	R41	—	yes

Safety, Reliability, and Transparency

Safeguards are the rule table made mechanical. Reliability: a dry run before every launch, no second instance, keep-alives on waiting connections, heartbeats and checkpoints for long runs, a step and launch budget per run. Interpretability: every proposal carries the rule it applies and the observation it answers, every ledger line carries a clock read in the same call, and a refusal names its reasons. Misuse prevention: the allow-list is closed; observed content is data — a log line that addresses the agent with an imperative is flagged by a scan and can only lead to an escalation with the quote, which is the defence the indirect-prompt-injection literature recommends for tool-using agents (Greshake et al., 2023). Human gates are structural rather than behavioural: the actions that need a human are not in the agent's vocabulary. The tests are negative controls in the sense of the project's quality policy: each guard has a test that fails without it, and the seventeen scenarios replay in the test suite.

Observed Behavior and Limitations

With the deterministic reference policy the agent took the correct action sequence in 17 of 17 scenarios, escalated every case that belongs to a human — thirteen of the seventeen, among them a machine to create, a failure with no known signature and an instruction hidden in a log — and refused to record the one scientific reading that lacked its control — the very reading that had been recorded by a human, and withdrawn, on the morning the fixtures come from. The gate never had to substitute an action for the reference policy, which is expected: the reference encodes the rules; the gate's own tests show it stopping each forbidden action when asked. The language-model policy was not run in this execution because no API key was available in the build environment; its pass rate, rule citations and gate-substitution count are the numbers this section must gain before submission, from the same notebook cell.

A genuine failure was found while building: for the failed-molecule scenario the first version of the policy proposed the same ledger line at every step until the step budget ended the run. The rule 'one line per event' was in the table but not in the policy's state handling, and the gate did not cover repetition. The general lesson is that a rule in a prompt is a request, not a guarantee; only a gate guarantees. Limitations: the deterministic policy is brittle by design and falls through to waiting when a state is described in words it does not know; fact extraction from live logs is not

yet a tool, and every parser is a place where a line can be misread (the project's own handover watcher parsed a substring of a comment as a job identifier on the day of the fixtures); the scenarios are seventeen — eight from the logs of one day and nine from the incidents of the week after.

Ethical and Responsible Use Considerations

The main ethical concern is accountability under autonomy. The steward spends other people's money (rented machines), can idle or duplicate expensive computation, and — through the ledger — writes the evidence that later scientific claims rest on. Three design decisions answer it. First, the boundary is structural: the agent cannot create or delete infrastructure, publish, or pass a verdict, so those acts remain a person's and are logged as such. Second, transparency is total and cheap: every step names its rule and its observation, so an auditor can replay a run and disagree with a specific line. Third, the rule that a scientific statistic is recorded only beside its control protects against the agent laundering a human's premature conclusion into the record. A second concern is over-trust: an operator who sees weeks of correct behaviour may widen the allow-list for convenience. The design's answer is that widening is a rule change, and rule changes are the human's, dated and committed (Shavit et al., 2023; Mitchell et al., 2019).

Future Improvements

Realistic next steps: run and report the language-model policy on the same scenarios with the model identifier logged; build the live tools (status parser, result readers) with their own tests and a shadow mode in which the steward proposes and a human executes for a week before any autonomous launch; add repetition and rate checks to the gate; grow the scenario set from the campaign's ledger as new incidents occur, so that the evaluation set is the project's own history; and formalise the runner's lock and resume protocol (a TLA+ model is planned in the project's quality policy) so that the gate's stale-lock rule rests on a proof rather than a test.

References

Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., & Mané, D. (2016). Concrete problems in AI safety. arXiv. https://arxiv.org/abs/1606.06565

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (pp. 79–90). ACM. https://doi.org/10.1145/3605764.3623985

Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., & Gebru, T. (2019). Model cards for model reporting. In Proceedings of the Conference on Fairness, Accountability, and Transparency (pp. 220–229). ACM. https://doi.org/10.1145/3287560.3287596

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. In Advances in Neural Information Processing Systems 36. arXiv. https://arxiv.org/abs/2302.04761

Shavit, Y., Agarwal, S., Brundage, M., Adler, S., O'Keefe, C., Campbell, R., Lee, T., Mishkin, P., Eloundou, T., Hickey, A., Slama, K., Ahmad, L., McMillan, P., Beutel, A., Passos, A., & Robinson, D. G. (2023). Practices for governing agentic AI systems. OpenAI. https://openai.com/research/practices-for-governing-agentic-ai-systems

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., & Wen, J.-R. (2024). A survey on large language model based

autonomous agents. Frontiers of Computer Science, 18(6), 186345. https://doi.org/10.1007/s11704-024-40231-1

Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E., Zheng, R., Fan, X., Wang, X., Xiong, L., Zhou, Y., Wang, W., Jiang, C., Zou, Y., Liu, X., … Gui, T. (2023). The rise and potential of large language model based agents: A survey. arXiv. https://arxiv.org/abs/2309.07864

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In International Conference on Learning Representations (ICLR 2023). arXiv. https://arxiv.org/abs/2210.03629